Single-source shortest paths

Part VI Graph Algorithms

Scope

Relaxation maintains upper bounds and predecessors reconstruct paths. We prove the distinct contracts of Bellman–Ford, DAG shortest paths, and Dijkstra, then use negative-cycle certificates, difference constraints, and rooted-tree verification to distinguish finite distance, unreachability, and unbounded cost.

Prerequisites

Edge relaxation, negative weights, and reachability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Maintain shortest-path and triangle-inequality invariants.
- Analyze Bellman–Ford, DAG shortest paths, and Dijkstra separately.
- Convert difference constraints to weighted directed graphs.
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

Three distance meanings; negative edges differ from cycles

Let $G=(V, E)$ be finite and directed, with $n \geq 1$, $s \in V$ and finite real edge weights. Path cost sums edge weights; a walk may repeat vertices and contain cycles. Define $\delta(s, v)$ as the infimum of costs of all s-to-v walks. It is ∞ if unreachable, $-\infty$ if a negative cycle can be visited on the way to v, and otherwise a finite minimum. The source's zero-length walk costs 0, so $\delta(s, s) \leq 0$.

If a walk to v has no usable negative cycle, deleting positive cycles improves its cost and deleting zero cycles preserves it. A simple shortest path of at most $n-1$ edges therefore exists. A negative edge alone does not make distances undefined. A negative cycle affects exactly vertices **reachable onward from that cycle when the cycle is reachable from s**; other branches can retain finite distances.

A single-destination problem becomes single-source after reversing all edges. A single-pair answer is one entry of the single-source result. Representing a negative undirected edge by two opposite arcs creates a negative closed walk; intuition about an isolated directed negative edge no longer applies.

Relaxation only improves tentative upper bounds

```
INITIALIZE(s):
    for v in V: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u, v)):
    if d[u]!=∞ and d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u, e.ID)
        return true
    return false
```

Induction on relaxations shows every finite d[v] is the cost of an actual s-to-v walk, hence $d[v] \geq \delta(s, v)$. Only strict improvements change parents. Parallel edges require IDs. Initialization is $\Theta(n)$, and each relaxation $O(1)$ under exact unit-cost arithmetic. Fixed-width implementations must avoid overflow and represent∞ separately. Clamping negative values to an arbitrary floor can destroy negative-cycle detection.

Once d reaches a finite δ, it cannot increase or fall below δ and stays fixed. Finiteness matters: $-\infty$ is not the ordinary finite output of a single relaxation.

Bellman–Ford operations, early stopping, and cost

```
BELLMAN-FORD(G, s):
    INITIALIZE(s)
    repeat at most n−1 rounds:
        changed=false
        for each e in E in a fixed order:
            if RELAX(e): changed=true
        if not changed: return d, parent,"no reachable negative cycle"
    for each (u, v) in E:
        if d[u]!=∞ and d[v]>d[u]+w(u, v):
            return "reachable negative cycle exists"
    return d, parent,"no reachable negative cycle"
```

The last scan tests for a cycle; a still-decreasing finite estimate is not delivered as a shortest distance. After an unchanged round, every reachable edge satisfies $d[v] \le d[u]+w$. Repeating the same scan cannot change anything, permitting early return. For n=1 the final test must still inspect self-loops and detect a negative one.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Bellman–Ford operations, early stopping, and cost (continued)

The full-round version takes $\Theta(n+nm)$, often written $\Theta(nm)$ when $m \geq 1$, $n \geq 2$. Early stopping has best case $\Theta(n+m)$ and the same worst case. Arrays d, parent use $O(n)$ extra space; input uses $O(n+m)$. Complete negative-cycle classification and certificates can use the DP below.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reverse edge order forces several propagation rounds

Vertices are (s, a, b, c, t, z). Scan the **fixed ordered edges** ct:3, at:8, ac:1, bt:12, bc:5, ba:−2, sa:4, sb:1. Each edge points from its first letter to its second. Vertex z is isolated.

Round end	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

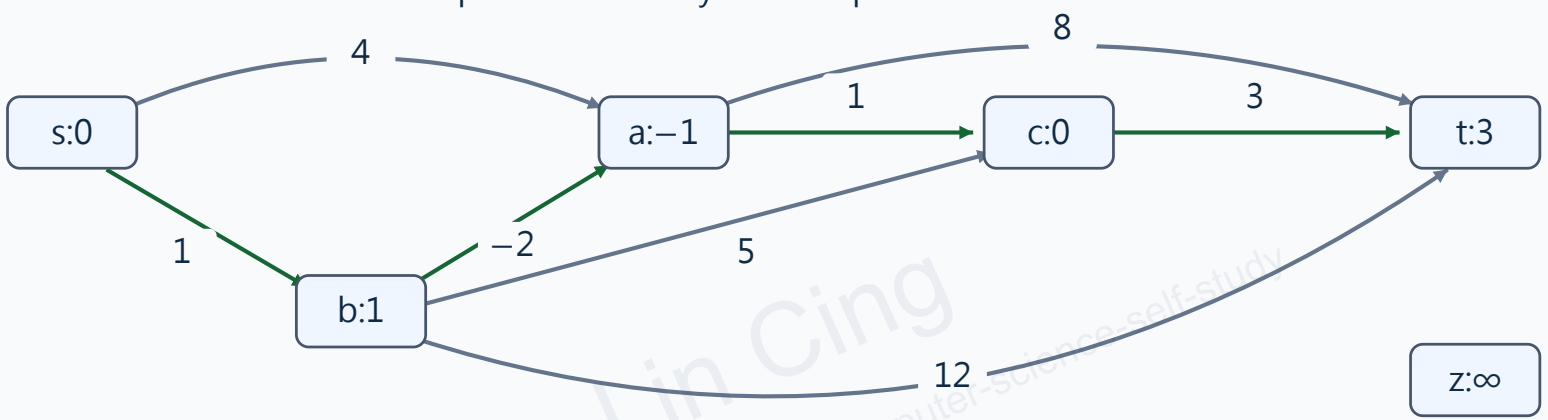
In round 2, old a=4 first gives t=12, c=5; only later does ba lower a to−1. In round 3, ct first gives 8, at lowers it to 7, then ac sets c=0. Only round 4 obtains t=3. Round 5 changes nothing. Final predecessors form s→b→a→c→t, cost 1−2+1+3=3.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Reverse edge order forces several propagation rounds (continued)

Bellman–Ford: one input and final predecessors

Nodes: final distances. Green: parent arcs. Gray: other input arcs.



Reverse-order round ends: a 4→−1; c ∞→5→0; t ∞→12→7→3.

Tree path s→b→a→c→t: 1−2+1+3=3. Vertex z is unreachable from s.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why n−1 rounds suffice and detect negative cycles

Without a reachable negative cycle, every reachable v has a simple shortest path $s=v_0,\ldots, v_k=v$ with $k\le n-1$. Round i scans (v_{i-1}, v_i), after round i−1 has already made $d[v_{i-1}]=\delta$. Convergence makes $d[v_i]=\delta$. Other same-round relaxations cannot spoil a true value. Thus n−1 rounds suffice. This proves the in-place algorithm covers at least i-edge paths; its state need not equal the DP minimum over paths with at most i edges.

If a negative cycle is reachable, all its vertices have finite estimates after n−1 rounds: reachability itself has a simple path of at most n−1 edges, without needing a shortest path. If no cycle edge could improve, summing $d[v]\le d[u]+w$ around it would cancel all distances and give $0\le w(\text{cycle})<0$. Thus some reachable edge must improve. Conversely, without such a cycle all distances are correct and triangle inequalities forbid improvement. These are both directions of the detection proof.

Reconstructing a certificate with an edge-count DP

Let $D_i[v]$ be the minimum cost of a walk with at most i edges, with $D_0[s]=0$ and all other entries∞.

$$D_i[v] = \min\left(D_{i-1}[v], \min_{(u,v)\in E}(D_{i-1}[u] + w(u,v))\right).$$

Each layer reads only the preceding layer. Record skip when retaining an old value, or the edge ID and preceding-layer vertex u when taking an edge. Every walk with at most i edges either has at most i−1, or loses its last edge to become one with at most i−1. Conversely every recurrence candidate is a legal walk, proving equality.

Compute through layer n. If $D_n[v]<D_{n-1}[v]$, backtracking n layers gives an **exactly n-edge** walk: fewer edges would already qualify at layer n−1. Decompose the walk into a simple path and cycles cut at repeated vertices. If every cycle were nonnegative, deleting them would produce an at-most-(n−1)-edge path of no greater cost, contradicting strict improvement. At least one extracted cycle is negative. Its ordered edge IDs and total weight form a checkable certificate.

Reconstructing a certificate with an edge-count DP (continued)

The full-history version takes $O(n+nm)$ time and $O(n^2+m)$ space. Two distance layers suffice for classification but do not retain this backtracking table. Do not claim the in-place algorithm's $O(n)$ auxiliary space for this version.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive −∞

Let seeds $B=\{v:D_n[v]<D_{n-1}[v]\}$. Search from B in the **original edge direction**, marking every reached vertex−∞. Other reachable vertices keep D_{n-1}; unreachable ones remain∞.

A seed's n-edge walk contains a negative cycle, so the seed and its descendants are affected. Conversely, every reachable negative cycle has finite entries in D_{n-1}. If it had no seed, each of its edges would satisfy $D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$; summing gives a contradiction. Every such cycle therefore has a seed, whose forward search reaches all affected descendants. The classification covers exactly the affected set, rather than only cycle vertices.

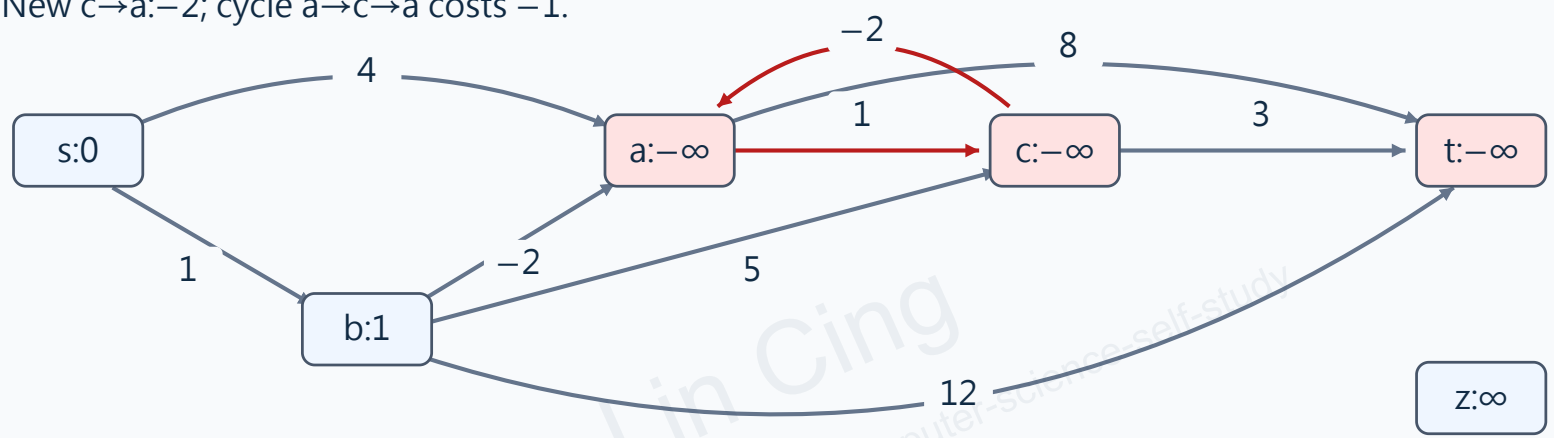
Add c→a:−2 to the earlier graph. Cycle a→c→a costs 1−2=−1. Distances on (s, a, b, c, t, z) become (0,−∞,1,−∞,−∞,∞). The cycle cannot return to s or b, which stay finite; z stays unreachable. An additional unreachable negative cycle must not cause a false single-source report.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exactly which vertices receive −∞ (continued)

A reachable negative cycle affects only forward descendants

New c→a:−2; cycle a→c→a costs −1.



Descendants a, c, t receive −∞; b=1, s=0. No cycle exit returns to s or b.

A negative cycle does not erase finite branches or unreachable z.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.2 Single-source shortest paths in directed acyclic graphs

Topological order delivers every candidate in time

```
DAG-SHORTEST(G, s):
    order=TOPOLOGICAL-SORT(G)
    if a cycle is found: reject this algorithm's input
    INITIALIZE(s)
    for u in order:
        for each edge e in Adj[u]: RELAX(e)
    return d, parent
```

Every s-to-v path is relaxed in its path order, so path relaxation gives δ at its end. Equivalently, induct on topological position: all incoming sources have been finalized and scanned before v is processed, leaving no future better candidate. Unreachable vertices retain∞. Negative edges are allowed because no cycle means no negative cycle. Sorting and scanning together take Θ(n+m), with O(n) auxiliary space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.2 Single-source shortest paths in directed acyclic graphs (continued)

The earlier graph is already a DAG. In order s, b, a, c, t, z, scanning s gives a4, b1; b gives a−1, c6, t13; a gives c0, t7; c gives t3. One pass matches four Bellman–Ford rounds. The difference is **topological vertex order**, not the cost definition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs

Negate every DAG weight and negate the resulting shortest cost to obtain its longest cost. Alternatively initialize $L[s]=0$, others $-\infty$, and use maximum relaxation. A cyclic graph cannot use the DAG algorithm unchanged; positive cycles make longest walks unbounded.

For vertex jobs of duration$[v] \geq 0$, an edge $u \to v$ requires u to finish before v starts. In topological order compute $F[v]=\text{duration}[v]+\max(\{F[u]:u \to v\} \cup \{0\})$. Backtracking a maximizing predecessor gives a critical path.

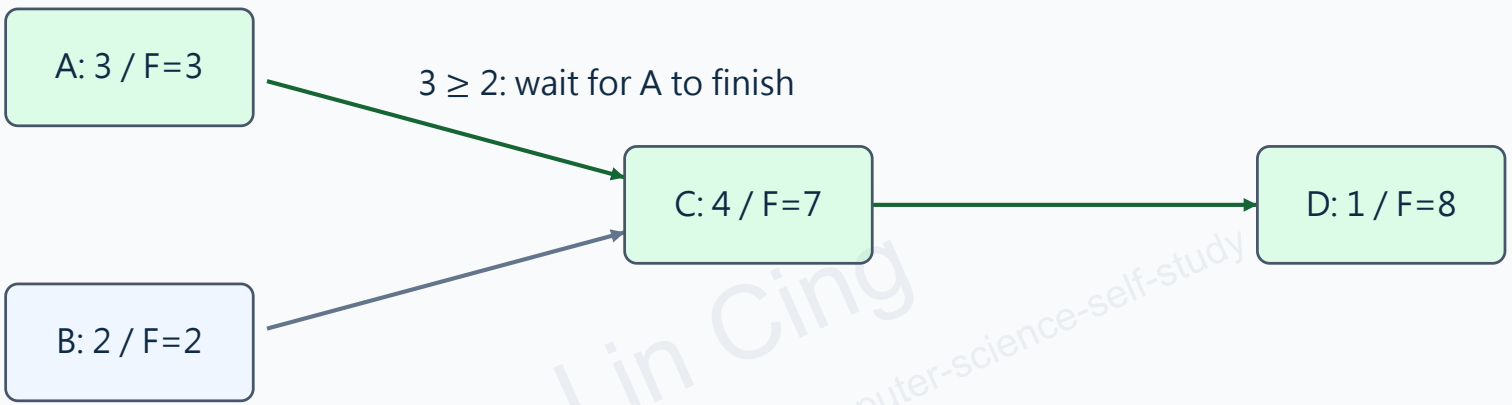
For A:3, B:2, C:4, D:1 with A→C, B→C, C→D, finishes are $F[A]=3$, $F[B]=2$, $F[C]=4+\max(3,2)=7$, $F[D]=1+7=8$. Critical path A, C, D costs 8. With unlimited processors and only these dependencies, starting each eligible job immediately attains this lower bound. Finite processors add resource constraints, so the critical-path bound alone is not a full schedule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Critical paths: sign reversal works directly in DAGs (continued)

Job DAG: the slower predecessor determines completion

Boxes: job / duration / earliest finish F. Green arrows: critical path.



F[C]=4+max(3,2)=7；F[D]=1+7=8。

Unlimited processors finish at 8. Total work 10 requires at least 10 on one processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extract the minimum tentative distance and never reopen it

Assume **every edge weight is nonnegative**. An indexed min-heap holds one item per unextracted vertex with key=d. Initialize d[s]=0, others∞, parent=NIL, inQ=true.

```
while Q is not empty:
    u=EXTRACT-MIN(Q); inQ[u]=false
    if d[u]==∞: stop; all remaining vertices are unreachable
    for e=(u, v) in Adj[u]:
        if inQ[v] and d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u, e.ID)
            DECREASE-KEY(Q, v, d[v])
```

Extracted vertices S have true distances. Unlike Prim, the candidate is **total path cost d[u]+w**. An extracted vertex is neither updated nor reinserted. A single-pair query may stop when the target is actually extracted, not when first discovered.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The precise use of nonnegative weights

Suppose the next finite extraction u is the first wrong one. On a true shortest path s→…→x→y→…→u, let y be the first unextracted vertex and x its extracted predecessor. The source is extracted first with d[s]=0. Otherwise x already has its true distance and has scanned xy, so convergence gives d[y]=δ(s, y).

The suffix y-to-u has nonnegative weight, giving δ(s, y)≤δ(s, u). Minimum extraction gives d[u]≤d[y]; the upper-bound property gives δ(s, u)≤d[u]. Thus

$$\delta(s,u) \le d[u] \le d[y] = \delta(s,y) \le \delta(s,u).$$

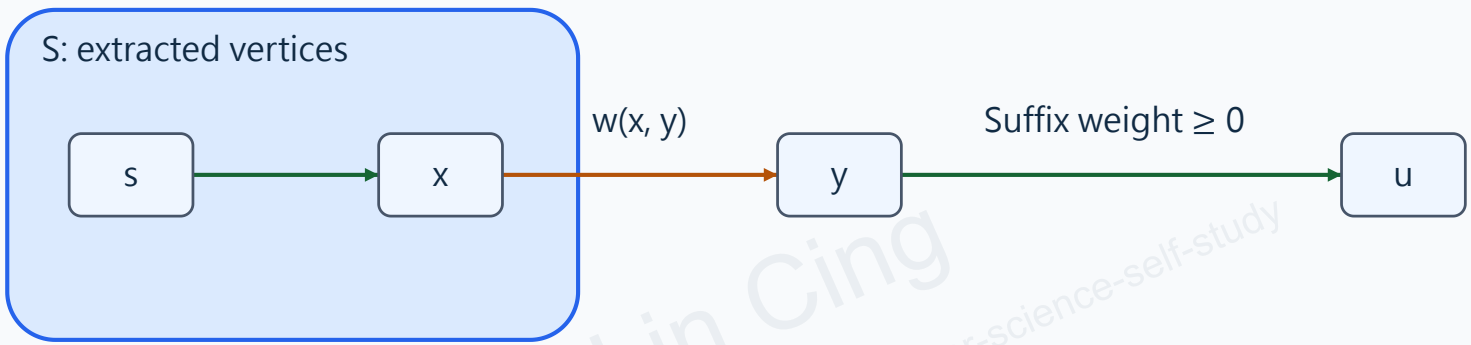
All inequalities are equalities, contradicting error. Zero edges and distance ties are allowed; strict increases are unnecessary. If an∞ item is extracted while a reachable vertex remains, the first unextracted vertex on its path would have a finite candidate, contradicting minimum extraction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The precise use of nonnegative weights (continued)

Dijkstra: bound u using the first unextracted vertex

Follow one true shortest path. Orange: the first arc leaving S.



Vertex x is finalized and xy scanned ⇒ $d[y]=\delta(s, y)$.

Minimum extraction and a nonnegative suffix give $d[u] \le d[y]=\delta(s, y) \le \delta(s, u)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical trace and a genuine negative-edge failure

Change only ba in the earlier example to+2. States after scans are

Extraction	Unextracted finite d(parent)	Finalized distances
s0	a4(s), b1(s)	s0
b1	a3(b), c6(b), t13(b)	s0, b1
a3	c4(a), t11(a)	also a3
c4	t7(c)	also c4
t7	none; z∞	also t7

The shortest-path tree is s→b→a→c→t, cost 1+2+1+3=7. Target estimates13→11→7 demonstrate why first discovery is insufficient.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Numerical trace and a genuine negative-edge failure (continued)

For a failure, use s→a:2, s→b:5, b→a:−4, a→t:2. After s, extract a2 and assign t4; extract t4 before b5, which would give a1 and true t3. The standard no-reopening algorithm retains wrong answers. Updating an extracted a without rescanning its outgoing edges also leaves t wrong. This graph is a DAG with no negative cycle. The failed premise is $\delta(s, b)=5>\delta(s, a)=1$, contradicting the proof's nonnegative-suffix comparison.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Queue, graph representation, and exact arithmetic costs

An indexed binary heap takes O(n) to build, O(n log n) for n extracts, and O(m log n) for at most one decrease per scanned edge, totaling O((n+m)log n), with n=1 handled directly. Fibonacci decreases take O(1) amortized, giving O(m+n log n). Handles locate heap items, and inQ is an O(1) array lookup.

Linear minimum-distance scanning with adjacency lists costs $O(n^2+m)$, simplifying to $O(n^2)$ for simple graphs. A lazy heap pushes all new candidates and can hold m items; it requires a separate O((n+m)log(m+1)) argument and O(n+m) space. The indexed n-item bound does not transfer automatically. The numerical example uses linear-key scanning and adjacency lists.

These bounds use a real-RAM model. With B-bit integer weights and no negative cycles, simple-path costs need O(B+log n) bits; exact addition and comparison costs must be included separately. Floating-point error or arbitrary epsilons can change equality and cycle decisions. Mathematical correctness alone does not establish numerical stability.

Derive the edge direction algebraically

Constraint $x_j - x_i \leq b$ corresponds to arc $i \to j : b$ because $x_j \leq x_i + b$. Each coefficient row has one $+1$ and one -1, with unrestricted real variables. Rewrite $x_j - x_i \geq b$ as $x_i - x_j \leq -b$. Split an equality into two opposite inequalities.

Add a super-source q with zero-weight arcs to every variable and no incoming arc, making all original negative cycles reachable. Without a negative cycle, set $x_i = \delta(q, i)$. Each edge's triangle inequality gives $x_j - x_i \leq b$. Conversely, summing constraints around a negative cycle cancels every variable and gives $0 \leq \Sigma b < 0$. Thus feasibility is equivalent to absence of negative cycles.

For n variables and m constraints, the augmented graph has $n+1$ vertices, $m+n$ edges, giving $O(n^2+nm)$ Bellman–Ford time. Instead initialize all original distances to 0, as though q's arcs have already been processed, and scan the original edges for at most $n-1$ rounds plus a test. Cost becomes $O(n+nm)$, including initialization when $m=0$.

Complete feasible and infeasible examples

Constraints $x_2 - x_1 \le -2$, $x_3 - x_2 \le -1$, $x_1 - x_3 \le 4$ give arcs $1 \to 2 : -2$, $2 \to 3 : -1$, $3 \to 1 : 4$. From all zero, the first scan gives $(0, -2, -3)$; $-3 + 4 = 1 \ge 0$ does not change x_1. The second round is unchanged.

Check each inequality: $-2 - 0 = -2 \le -2$; $-3 - (-2) = -1 \le -1$; $0 - (-3) = 3 \le 4$. Adding 3 gives $(3, 1, 0)$, also feasible because differences are unchanged. The feasible region is invariant under translation along the all-ones vector and has no fixed origin.

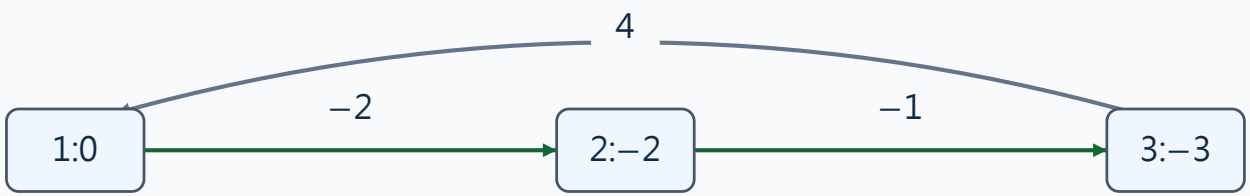
Changing the last constant to 2 makes the sum $0 \le -2 - 1 + 2 = -1$, proving infeasibility. The negative cycle is $1 \to 2 \to 3 \to 1$. Failure of one proposed vector alone would not prove absence of every feasible vector.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete feasible and infeasible examples (continued)

Difference constraints: a feasible vector and a cycle contradiction

Arc i→j:b means xj−xi≤b. Node values are a feasible vector x.



Differences: −2−0=−2; −3−(−2)=−1; 0−(−3)=3≤4.

Change the last weight 4 to 2: summing gives 0≤−2−1+2=−1, a contradiction.

Original cycle weight 1: feasible. New weight−1: infeasible. Translation preserves differences.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Extensions that retain the proof

Equality $x_j - x_i = b$ gives $i \to j : b$ and $j \to i : -b$. Together they force equality. To express $x_i \leq c$ or $-x_i \leq c$, introduce a reference r and respectively $r \to i : c$ or $i \to r : c$. After solving, translate all coordinates so $x_r = 0$ before treating them as absolute bounds. Translation must preserve this reference.

Integer right sides and integer-zero initialization make Bellman–Ford's additions and minima produce an integer feasible solution. If **all** variables must be integers but right sides are real, replace each b by $\text{floor}(b)$: integer differences make the constraints equivalent. Then test for negative cycles. This rowwise rounding is not automatically valid when only some variables must be integers.

Under the normalization $d \leq 0$, shortest distances form the coordinatewise greatest feasible vector. Any feasible x satisfies $x_v \leq$ the cost of every q-to-v path by repeated substitution, hence $x_v \leq \delta(q, v)$. This conclusion includes $x_i \leq 0$. With unrestricted translation in the original system, Σx_i can increase arbitrarily.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimal substructure, triangle inequality, and convergence

Assume no negative cycle reachable from s. A subpath of a finite shortest path must be shortest: replacing a more expensive subpath gives a cheaper s-to-v walk, a contradiction. Appending $u \to v$ to a shortest s-to-u path gives $\delta(s, v) \leq \delta(s, u) + w$. An unreachable u makes the right side ∞ and imposes no additional restriction.

Initialization and the actual-walk interpretation prove the upper-bound property. An unreachable vertex cannot become finite without an actual walk. Immediately after RELAX(u, v), $d[v] \leq d[u] + w$, but **only at that instant**: a later decrease in d[u] can require rescanning this edge.

If u already has its true distance and $u \to v$ lies on a shortest path, relaxation gives $d[v] \leq \delta(s, u) + w = \delta(s, v)$. The upper bound gives the reverse inequality, so v becomes permanently correct. This is convergence. Starting with s and applying convergence in shortest-path edge order fixes one vertex after another. Interspersed relaxations only lower upper bounds and cannot spoil correct values, establishing path relaxation.

Parent acyclicity follows from update times

Suppose a strict relaxation $u \to v$ first creates a parent cycle. Before it, the parent subgraph has a path $v \to \dots \to u$. For every old parent arc $x \to y$, the last assignment of parent[y] made $d[y] = d[x] + w$. Since y has not been reassigned and x can only decrease, immediately before this update $d[y] \geq d[x] + w$. Summing the old path gives $d[u] \geq d[v] + w(v \to \dots \to u)$.

The new strict update requires $d[v] > d[u] + w(u, v)$. Combining gives a negative total cycle weight. All its vertices are finite and reachable from s, contradicting the assumption. A negative self-loop follows directly from the same strict improvement. Thus the parent subgraph remains acyclic.

Every finite $v \neq s$ received a non-NIL parent by relaxation. The source retains $d[s] = \delta(s, s) = 0$ and parent NIL. Parent vertices of finite vertices are also finite. In a finite acyclic graph, backtracking must terminate at the only parentless finite vertex s. Each vertex has one parent, giving a rooted tree on all discovered finite vertices.

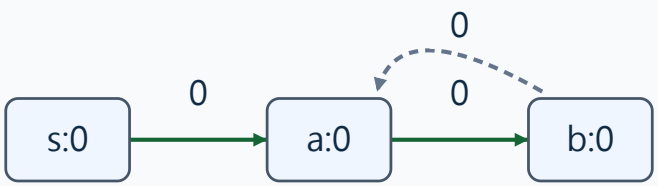
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Parent acyclicity follows from update times (continued)

Zero edges: equal distances still need rooted parents

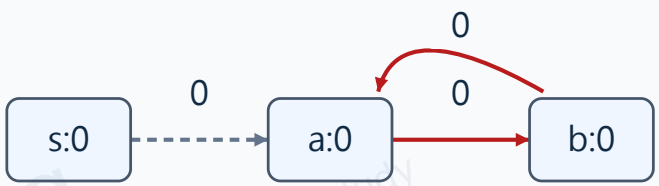
Original arcs s→a:0, a→b:0, b→a:0. All distances are 0 on both sides.

Valid parents: rooted paths from s



parent[a]=s · parent[b]=a

Invalid parents: a, b form a cycle



parent[a]=b · parent[b]=a

Green: valid tree. Red: invalid cycle. Gray dashed: unselected original arcs.

Tightness checks cost. Search parent children from s to verify connectivity and acyclicity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correct distances and valid parents give a shortest-path tree

Once finite d equals δ, the last-update argument for every current parent arc u→v gives d[v]≥d[u]+w. The triangle inequality gives the reverse after substituting d=δ, hence equality. Summing on the s-to-v tree path telescopes to path cost d[v]−d[s]=δ(s, v). Unreachable vertices retain∞ and NIL parents, completing all shortest-path-tree requirements.

Do not justify this by strictly decreasing predecessor distances. Zero edges permit equality and negative edges can decrease forward distances. Acyclicity depends on the strict-update-time argument. Choosing arbitrary tight predecessors afterward can form a zero cycle. In s→a:0, a→b:0, b→a:0 all distances are 0; parent[a]=b, parent[b]=a makes both edges tight without any root path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A linear-time certificate verifies optimality too

Given d and parent edge IDs, compute the actual reachable set R from s. Require finite d in R, d[s]=0, and∞ with NIL parents outside R. For every v∈R−{s}, verify that its parent is an actual input arc u→v with d[v]=d[u]+w. Search the parent-children graph from s, visiting exactly R without repeats, and require parent[s]=NIL. Finally scan all input arcs from reachable u and check d[v]≤d[u]+w.

The tight rooted tree realizes a path of cost d[v], proving δ(s, v)≤d[v]. Summing edge inequalities along any s-to-v walk gives d[v]≤its cost, hence d[v]≤δ(s, v). Both directions establish equality. A reachable negative cycle would contradict the summed inequalities and is rejected. Verification takes Θ(n+m) time, O(n) space without rerunning Bellman–Ford or Dijkstra.

Edge inequalities alone allow unrealistically small values with no realizing paths. Tight parents alone allow zero cycles or overlook cheaper input arcs. Each certificate component proves a separate direction.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 24.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study